

- * Latest version of Java - 24.
- * Java-8 launched in 2014.
- * There were a breaking changes in Java-8.
- * Functional Programming approach.
 - Coding in less number of lines.
 - This made more readable.

* Features of Java-8:

1. Functional Interface
2. Default and Static methods in Interface. (we can write implementation here).
3. Lambda Expression (→) [works only with functional interfaces]
4. Method References
5. Optional Class.
6. Stream APIs
7. LocalDateTime class.
8. Collectors

1. Functional Interface:

- * It has only one public abstract method.
- * It is a type of interface.
- * We need to write @FunctionalInterface above interface.
- * This annotation is not mandatory (@FunctionalInterface).
- * ex: Runnable interface (used in Threads). → It has only one method in it i.e. run().

Ex: package com;

@Functional Interface,

public interface Functional Interfaces {

public void sayHi(); ✓

default void hi() { ✓

syso("Hi--");

}

package com;

@Functional Interface.

public interface Interface2 extends Functional Interfaces {

// *** This Interface2 gets inherited method from parent class.

Due to this we need not to write abstract method.

}

Marker Interface:

* Interface with zero methods.

* This is an indication to JVM to give some behaviour.

* Ex: Cloneable Interface, **S**erializable Interface.

2. Default & Static methods in Interfaces:

* Due to this, interfaces are not 100% abstraction.

* These methods can take implementation in Interfaces.

* Here, default is not an access modifier.

* We can write multiple default methods & we can override it in child class.

* We can write default method in function interface with abstract method.

Ex:
* Interface1 → sayHi (default) Interface2 → sayHi (default)

Class implements Interface1, Interface2 {

→ [It again enters into diamond problem (ambiguity issue). We need

to override that method in child class to ignore this issue.]

}


```

Ex: package com;
    @FunctionalInterface
    public interface FunctionalInterfaces {

        public void sayHi();
        default void hi() {
            syso ("Hee---");
        }
    }

```

```

package com;

public interface Interface2 {

    public void sayBye();
    public default void sayHi() {
        syso ("Hi---");
    }
    default void defaultMethod() {
        syso ("From Interface2 default method");
    }
}

```

[Normal Interface] →

```

package com;

public class Impl implements Interface2 {

    @Override
    public void sayBye() {
        syso ("Bye---");
    }
}

```

```

    @Override
    public void sayHi() {
        syso ("Hi---");
    }
    @Override
    default void sayHi defaultMethod() {
        syso ("Hi---");
    }
}

```

```

package com;

public class Test {

    public static void main (String args[]) {
        Interface2 if2 = new Impl();
        if2.sayHi();
        if2.sayBye();
        if2.defaultMethod();
        if2.sayHi();
    }
}

```

Output :

Hi---
Bye ---
From Interface2
Default Method
child Hi---
Hi---
Hee---

Static methods in Interface:

```
public Interface Interface2 {  
    static void StaticMethod() {  
        Syso("Static method");  
    }  
}
```

- * We can't override static methods.
- * These methods cannot be overridden in child class.
- * Here there is no method hiding also.
- * Here we can't able to call that method with child object.

PSVM {

Classname.StaticMethod();
Interface

Interface2.StaticMethod();

if2.StaticMethod(); [we can't call with object].
(error)

class: 40

13/06/2025

* Anonymous Inner Class:

- * This is a implementation without any name and provides implementation to that class.

Ex: package com;
abstract
public class Vehicle {
 public ~~void~~ abstract void start();
 public abstract void stop();
}

// This anonymous inner class works for abstract & normal methods also.

- * We can access methods only from parent class when we hold object with parent class.
- * This works only temporarily. We cannot access this class in other class.

package com;

public class Bike extends Vehicle {

@Override

public void start() {

 System.out.println("Started Bike");

}

@Override

public void stop() {

 System.out.println("Stop Bike");

}

package com;

public class Test {

 public static void main(String[] args) {

 Bike bike = new Bike();

 bike.start();

 bike.stop();

 Vehicle lorry = new Vehicle() { ⇒ Anonymous class.

 @Override

 public void start() {

 System.out.println("Started Lorry");

 }

 @Override

~~lorry.start();~~

 public void stop() {

 System.out.println("stop lorry");

 }

 }

 ** {;

 lorry.start();

 lorry.stop();

 }

}

Output:

Started Bike

Started Lorry.

Stop Bike

Stop Lorry

package com;

public class ThreadTest {

public static void main {

Thread t₁ = new Thread();
t₁.start();

Thread t = new Thread(t₁);
t.start();

Runnable t₂ = new Runnable() {

@Override

public void run() {

System.out.println("From thread 2");

}
} Class Thread1 extends Thread {

@Override

public void run() {

System.out.println(Thread.currentThread().getName());

System.out.println("From Thread");

}

}

Class Thread1 implements Runnable {

@Override

public void run() {

System.out.println(Thread.currentThread().getName());

System.out.println("From Thread");

}

}

Anonymous Inner class.

Thread th2 = new Thread(t₂);
th2.start();

Output:

Thread-0

From Thread.

From Thread2

3. Lambda Expressions:-

- * This works with the help of "→".
- * This works only on functional interfaces. only.
- * This doesn't have access modifiers, return type, method name.

⇒ public void sayHi() {

}

Syntax:-

Implementation.

⇒ () → { }

Ex.

```
package com;
@FunctionalInterface
public interface Vehicle {
```

```
    public abstract void start();
```

```
}
```

```
package com;
```

```
public class Bike implements Vehicle {
```

```
@Override
```

```
    public void start() {
```

```
        sysout("Started");
```

```
    }
```

```
package com;
```

```
public class Test {
```

```
    psvm {
```

```
        Vehicle bike = new Bike();
```

```
        bike.start();
```

```
Vehicle car = ( ) → { sysout("started car"); };
```

```
car.start(); ( ) → sysout("started car");
```

⇒ [we can write without brackets also.]
(with single line only)

Case (i):-

Created new file (Bike).

Overridden the method inside it.

Case (ii):-

Didn't create a separate class

Override start method inside anonymous class.

Case (iii):-

Didn't create separate class,

Didn't override that method (with →)

Runnable t3 = () → Syso("From Thread 3");

Thread th3 = new Thread(t3);

th3.start();

(8)

Thread th3 = new Thread(() → Syso("From Thread 3");)

package com;

public interface Calculator {

public void add(int a, int b);

int

package com;

public class Test {

psvm {

Calculator calc = (int a, int b) → {{ Syso(a+b) }; };

calc.add(10, 30);

(8)

(a, b)

output: 40

Calculator calc = (a, b) → { return a+b; };

int result = calc.add(10, 30);

Syso(result);

}

}

16/06/2025

class: 4

COMPARABLE & COMPARATOR (Interface)

- * sorts an array with user defined class generics/custom classes.
- * To sort the data of user defined class, we use comparable & Comparator.
- * Comparable comes from java.lang.
- * Comparator comes from java.util.
- * Comparable is an Interface (Functional Interface).
- ⇒ public int compareTo(); → has only one method in it.
- It is used to compare

[ctrl+shift+t]

↓
to open class explanation.]

- * It internally calls compareTo() method.
- * It allows only natural ordering (will not change order).
- * It allowed only at class level.

Ex: package com;

```

public class Student implements Comparable<Student> {
    int id;
    String name;

    public Student() {
    }

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

```

getters & setters & toString method.

@Override

```

public int compareTo(Student student o) {
    return this.id - student.id;
}

```

* return (this.id == student.id) ? 0 : (this.id > student.id) ? 1 : -1;

[// It compares with the help of positive, negative numbers and zero.]

current references

// It internally calls Student1.compareTo(Student2).
Student2.compareTo(Student3).

// to compare with names

* return (-this.name.compareTo(Student.name));

[∵ String has compareTo() method.]

package com;

public class Test {

psvm {

List < Student > students = new ArrayList < student > ();

students.add (new Student (10, "xxxx"));

students.add (new Student (2, "yyyy"));

students.add (new Student (30, "zzzz"));

students.add (new Student (4, "tttt"));

syso (students);

Collections.sort (students);

syso (students);

[∴ If we want to sort
we need to implement
comparable interface].

Output:

[students [id = 10; name=xxxx ---]

[students [id = 2, name = yyyy] [id = 4, name = "tttt"] ---

student [id = 4, name = tttt] student [id = 10, name = "xxxx"] ---

Comparators:

* It ~~allows~~ allows custom order (we can maintain multiple orders):

* We need to implement in class level.

* It provides two parameters.

* ~~str~~

Ex:

package com;

public class Employee {

int empId;

String domain;

String empName;

Constructors, getters & setters, toString method.

Psvm {

Comparator < Employee > companyComparator = new Comparator {

@ Override

public int compare(Employee emp1, Employee emp2) {

return emp1.domain.compareTo(emp2.domain);

}

};

Collections.sort(employees, companyComparator);

(8) Sysout(employees);

Lambda Expression:

Comparator < Employee > idComparator = (emp1, emp2) ->

{ Integer.compare(emp1.empId, emp2.empId); }

Collections.sort(employees, idComparator);

sysout(employees);



(8)

Collections.sort(employees, (emp1, emp2) -> emp1.domain.compareTo(emp2.domain));

Sysout(employees);

(8)

TreeSet < Employee > empSet = new TreeSet < Employee > ();


```
package com;
```

```
public class Test {
```

```
    Psum {
```

```
        List < Employee > employees = new ArrayList < Employee > ();
```

```
        employees.add (1, "Java", "Ravi");
```

```
        employees.add ( new Employee (123, "Java", "Ravi") );
```

```
        employees.add ( new Employee (234, "Java", "Harish");
```

```
        employees.add ( new Employee (456, "Java", "Gayatri");
```

```
        employees.add ( new Employee (109, "Java", "Fayaz");
```

```
        Syso (employees);
```

```
        Collections.sort (employees, new Id Comparator ());
```

```
        Syso (employees);
```

```
        Collections.sort (employees, new Name Comparator ());
```

```
        Syso (employees);
```

```
package com;
```

```
public class Id Comparator implements Comparator < Employee > {
```

```
@Override
```

```
public int compare (Employee emp1, Employee emp2) {
```

```
    return Integer.compare (emp1.empId, emp2.empId);
```

```
// It compares two integer variables.
```

```
} // Integer has no compare method)
```


package com;

public class NameComparator implements Comparator < Employee > {

@Override

public int compare (Employee emp1, Employee emp2) {
return emp1.empName . compareTo (emp2 . empName);

} [(emp1, emp2) → emp1.empName . compareTo (emp2.empName);]
(lambda Expression) (8)

id Comparator

(8) TreeSet < Employee > empset = new TreeSet < Employee > ();

empset . add (new Employee (234, "Java", "Fayaz");

empset . add (new Employee (123, "Python", "Ravi");

empset . add (new Employee (456, "C", "Gayatri");

empset . add (new Employee (109, "Ruby", "Harish");

System.out.println (empset); [TreeSet internally works with sorting only]

class: 42

STREAMS (class) APIs

17/06/2025

* It is a sequence of operation happening in a pipeline.

* It uses with collections.

1. Intermediate Operation

2. Terminal Operation

* Streams donot modify our actual list (stored in pipe line).
Collection

Intermediate Operations:

* It takes out the data and do some operations on it (like filter, modify, sort etc).

* We can use multiple Intermediate Operations.

Terminal Operations:

* After some operation, the data collected using Terminal operations using collectors. [Collect (collectors . toList/set());]

* We can't use two terminal operations at a time.

* We can't use other operations after terminal operations.

Constructors, Getters & Setters, toString().

- `collect (Collectors.toList());` // Terminal operation.

// to collect data after filter.

syso(even); → output: [10, 20, 30, 2, 4]

```
List<Employee> employees = new ArrayList<Employee>();
```

```
employees.add(new Employee(1, "Fayaz", 1000));
```

```
employees.add(new Employee(2, "Ravi", 100000));
```

```
employees.add(new Employee(3, "Sai", 200000));
```

```
employees.add(new Employee(4, "SriRam", 350000));
```

employees

- stream()

- filter(emp → emp.salary > 100000)

- forEach(emp → syso(emp));

Output:

Employee [id = 2, name = Ravi, Salary = 100000]

Employee [id = 3, name = Sai, Salary = 200000]

Employee [id = 4, name = SriRam, salary = 350000]

// sort the List using Java 8.

```
List<Integer> sortedList = nums
```

nums.

- stream()

- sorted()

- toList(); // to collect data without using

- collect(Collectors.toList());

syso(~~num~~ sortedList); output: [2, 4, 10, 17, 20, 30].

// Employee

- sorted(Comparator.reverseOrder());

output: [30, 20, 17, 10, 4, 2]

```
List<Employee> sortedEmployees = employees
```

employees

- stream()

- // sorted() → ~~it~~ cannot sort directly custom class.

- // toList()

- sorted((emp1, emp2) → Integer.compare(emp1.salary, emp2.salary)).

- toList();

syso(sortedEmployees);

// to increase salary by 5000/- — modify data — we use

employees

• `map()`, method
(method in stream)

• `stream()`

• `map(emp) → emp.salary += 5000)`

• `forEach(emp → sysout(emp));`

output:

6000

115000

205000

355000

// to print Employee

Employee

• `map(emp) → new Employee(emp.id, emp.name, (emp.salary += 5000))`

List < Employee > names = employees

employees

• `stream()`

• `filter(emp → emp.name.startsWith("S"))`

• `toList()`

`sysout(names);`

// Remove duplicates from the list using Java-8.

List < Integer > unique = nums

nums.

[10, 10, 10, 21, 30, 21, 4, 17]

• `stream()`

• `distinct()` → removes all the duplicates.

• `toList()`

• `sysout(unique);` → output: [10, 21, 30, 21, 4, 17]

// Find the highest value from the list.

Integer Integer = nums;

nums

• `stream()`

• `sorted(Comparator.reverseOrder())`

• `findFirst()` → gives 1st element in the list.

• `get()`

`sysout(integer);` → output: 30

// Find the second highest value which has duplicates.

[10, 5, 10, 15, 2, 2, 20, 20, 17]

Integer secondHighest = nums

nums

• `stream()`

• `distinct()`

• `sorted(Comparator.reverseOrder())`

• `skip(1)`

• `findFirst()`

• `get()`

`sysout(secondHighest);` →

output: 17

Ex:

package com;

public class HashMapFunctions {

Psrm {

HashMap <Integer, String> names = new HashMap <Integer, String> ();

names.put (1, "Ravi");

names.put (2, "Fayaz");

names.put (3, "Nani");

// names.put (

(∴ In map, direct stream is not possible)

// to get both values & keys.
names.

.entrySet ();

.stream ();

.forEach (entry → syso (entry.getKey () + "-" + entry.getValue ());

// to get only keys,

names

.keySet ();

.stream ();

.forEach (key → syso (key));

output:

1 - Ravi

2 - Fayaz

3 - Nani

// to get only values,

names

.values ();

.stream ();

.forEach (val → syso (val));

output:

1

2

3

output:

Ravi

Fayaz

Nani

// collect keys which have id >= 2 from a map

Map <Integer> string > filterMap = names

names

.entrySet ();

.stream ();

.filter (entry → entry.getKey () >= 2)

.collect (Collectors.toMap (entry.getKey (),

entry → entry.getKey (), entry → entry.getValue ());

syso (filterMap),

(8)

output:

{2 = Fayaz;

3 = Nani }

forEach (entry → syso (entry.getKey () + "-" +

entry.getValue ());

// print all keys which are having course1
map →

```
Map <String, String> courses = new HashMap <String, String>();
```

```
courses.put("string1", "course1");
```

```
courses.put("string2", "course2");
```

```
courses.put("string3", "course1");
```

```
courses.put("string4", "course1");
```

```
List <String> filteredKeys = courses
```

```
courses
```

```
    .entrySet()
```

```
    .stream()
```

```
    .filter(entry → entry.getValue().equals("course1"))
```

```
    .map(entry → entry.getKey())
```

```
    .toList()
```

```
    .forEach(System.out::println);
```

output:

[string4, string3, string1]

```
// .collect(Collectors.toMap()
```

```
    entry → entry.getKey(), entry → entry.getValue());
```

→ to get both key & value pairs.

// Count even numbers in a list.

```
List <Integer> list = new ArrayList <Integer> (Arrays.asList(10, 13, 17, 20,
```

```
24));
```

```
long count = list
```

```
    .stream()
```

```
    .filter(n → n % 2 == 0)
```

```
    .count() → return long type values.
```

```
    .forEach(System.out::println);
```

// sum of nums in a list.

```
int sum = list
```

```
list
```

```
    .stream()
```

```
    .mapToInt(n → n)
```

```
    .sum()
```

```
    .forEach(System.out::println);
```

(sum()) → doesn't work

on Integers, it works

only on primitive data

types

output: 84

// Optional Double average = list double.asDouble() = list
list // to calculate average of list.

• stream()

// average() → It doesn't work on wrapper classes.

• mapToInt(n → n)

• average();

• getAsDouble();

~~System.out.println(average.getAsDouble());~~ **output: 16.8**

System.out.println(asDouble);

// Max & Min

*** int max = list.

list

• stream()

• mapToInt(n → n)

• max()

• getAsInt();

System.out.println(max);

→ **output: 24**

*** int min = list

• stream()

• mapToInt(n → n)

• min()

• getAsInt();

System.out.println(min); → **output: 10**

// flatMap() → merges multiple collections into a single collection.

map() → to modify some data.

Ex:

List<List<String>> words = new ArrayList<List<String>>();

words.add(Arrays.asList("Hi", "Eye"));

words.add(Arrays.asList("one", "Two")); **output:** [[Hi, Eye], [one, Two]]

System.out.println(words);

words.add(Arrays.asList("Gm", "Gn"));

words list < String> list 3 = words

- stream()

- flatMap(list2 → list2.stream()) // to merge multiple lists into single list.

- toList();

sysout(list3);

output:

[Hi, Bye, One, Two, Gm, Gn]

* Parallel Stream();

* It acts like a thread (multithreading concept).

* Order is not maintained (when we use huge data).
Works faster than stream().

* ex: list < Integer> list2 = new ArrayList<Integer>(Arrays.asList(10, 13, 17, 20, 24));

list2

- parallelStream()

- forEach(n → sysout(n));

output:

17

24

20

13

10

list2

- stream()

- limit(2) → limits number of elements to return.

- forEach(n → sysout(n));

output:

10

13

boolean allMatch = list2

list2

- stream()

- allMatch(n → n > 20) → returns boolean values.

sysout(~~list2~~ allMatch);

output: false.

Grouping By Functions

* By using this we can count frequency.

Ex:

package com;

```
public class StringFrequency {
    Psvm {
```

```
List<String> fruits = new ArrayList<String> (Arrays.asList(
```

```
"apple", "guava", "apple", "mango", "mango");
```

```
Map<String, Long> frequency = fruits
```

```
fruits
```

```
.stream()
```

```
.collect(Collectors.groupingBy (Function.identity())
```

```
↓
```

gives frequency

```
Collectors.counting())
```

```
↓
```

(it will count) - value

Output:

```
{ apple=2, guava=1, mango=2 }
```

// Character frequency with String.

String fruit = "Apple";

Map<Character, Long> CharFrequency = fruit:

```
fruit // .toLowerCase() → to get all chars into lower case.
```

```
.chars() → [this will convert String into a int
```

```
Stream (ASCII value) [because
```

```
Streams work on collections]
```

```
.mapToObj(c → (char)c) → [to change int to char]
```

(Explicit type casting)

```
.collect(Collectors.groupingBy(Function.identity(),
```

```
Collectors.counting()));
```

```
sysout(CharFrequency);
```

Output: { p=2, A=1, e=1, l=1 }

int[] a = {1, 2, 3, 4, 5};

Arrays.

```
.stream(a)
```

```
.filter(n → n%2 == 0)
```

```
.forEach(n → sysout(n);
```

(8)

```
.forEach(System.out :: println);
```

Output: 2

METHOD REFERENCES

- * Ex: Employee has `getId()` method.
We call this method as `(emp)obj.getId()`
We can call it as, `Employee::getId()` → Method references.
`System.out::println`;

* Reduce() → Terminal Operation.

- * `reduce()` - It reduces into a single value.
- * It works as a loop & reduces the list/Array into a single value. It takes two parameters → 1. Initial value
2. Actual logic.

- Ex: `package com;`
`public class Test {`
 `psvm {`

 `List<Integer> nums = new ArrayList<Integer>(Arrays.asList(`
 `1, 2, 3, 4, 5));`

 `Integer sum = nums`
 `nums`
 `.stream()`
 `.reduce(0, (a, b) → a + b);`
 `syso(sum);`
 `Integer product = nums`
 `nums`
 `.stream()`
 `.reduce(1, (a, b) → a * b);`
 `syso(product);`

 `List<String> words = new ArrayList<String>(Arrays.asList("Fayaz",`
 `"is", "a", "Java", "Trainer"));`

 `String finalString = words`
 `.stream()`
 `.reduce(" ", (a, b) → a + " " + b);`
 `syso(finalString);`
 `Fayaz is a JavaTrainer.`
 `(finalString.trim()) → removes spaces.`

Optional Class

- * It creates wrapper. It will not give direct null value.
- * It directly doesn't provide value, it stores into Optional.
Optional.of()
- checks by using
(stores values)
- * It is used to avoid "Null pointer Exception".

package com;

```
public class Employee User {
```

```
    int userId;
```

```
    String email;
```

```
    public User() {
```

```
    }
```

Constructor & Getters & Setters & toString().

```
    public String getUserEmailById (int id) {
```

```
        return null; this.email; (8) return Optional.of(email);
```

```
        if (id == 10) {
```

```
            return Optional.of(email);
```

```
        } else {
```

```
            return Optional.empty();
```

```
        }
```

package com;

```
public class Test {
```

```
    Psvm {
        User user2 = new User (10, "Fayaz");
        User user = new User (20, null);
```

```
String email = user.getUserEmailById (10);
```

```
System.out.println(email.length());
```

```
System.out.println(email.get());
```

Null pointer Exception

(We can't do any operations on null).

```
email.ifPresent() (8)
```

```
email.ifPresentOrElse(email -> System.out.println(email), () -> System.out.println("No Value"));
```

```
Optional<String> email = user.getUserEmailById(10);
```

(20)

output: Fayaz

No Value.

```
Optional.empty() = Optional<Object>.empty()
```

```
Optional.of("FLM") = Optional.ofString if
```

```
System.out.println(email.get());
```


Optional < Object > of Nullable, = Optional.of(Nullable (null));
 of Nullable . if Present (n → Syso(n));
 ↓
 It will avoid null values.
 (it doesn't print anything)

// Optional < User > user = findById(10);

Optional < String > name = Optional.of("Fayaz");

~~name~~ Optional < Integer > map = name

.map (n → n.length()) (8) .map (string :: length);

↑
 Syso (map.get());

output: 5

20/06/2025

class-45

Local DateTime Class

ex:

package com;

public class DateTimeJava8 {

psvm {

class

* LocalDate today = LocalDate.now(); → we can get today's date.

Syso(today); → output: 2025-06-20

* LocalDate of

= LocalDate.of (2000, 05, 27);

Syso(of); → output: 2000-05-27

* LocalDate yesterday = ~~Local~~ LocalDate.parse ("2025-06-19");
 Year Month Date

Syso(yesterday); output: 2025-06-19

Syso (today.getYear()); ⇒ output: 2025

Syso (today.getMonth());

June

Syso (today.getDayOfMonth());

20

Syso (today.getDayOfWeek());

FRIDAY

Syso (today.getDayOfYear());

171

2025-06-21

SATURDAY

LocalDate tomorrow = Syso (today.plusDays(1));

Syso (tomorrow);

Syso (tomorrow.getDayOfWeek());

* LocalDate yesterday = today.minusDays(1); **output:**
 syso(yesterday.getDayOfWeek()); THURSDAY
 syso(yesterday.isBefore(today)); true.
 syso(today.isBefore(yesterday)); false
 syso(today.isAfter(yesterday)); true
 syso(today.isLeapYear()); false

* LocalDate leap = LocalDate.of(2024, 01, 01);
 syso(leap.isLeapYear()); 20 | JUNE | 2025
 // to change format of date. "dd/MM/yyyy"

* DateTimeFormatter pattern = DateTimeFormatter.ofPattern("dd/MM/yyyy");

syso(today); **output:** 2025-06-20
 syso(today.format(pattern)); 20 | 06 | 2025
 syso(today.getMonth()); JUNE
 syso(LocalDate.of(2025, 8, 25).format(pattern)); 25 | August | 2025

[Octal Numbers]
 ↓
 08, 09 (issue)
 It will not allow in Java

* // Gives current time & date.

LocalDateTime now = LocalDateTime.now();

syso(now);

output:

2025-06-20 T 20:15:35.

syso(now.getHour());

20 (hours)

syso(now.getMinute());

17 (minute)

syso(now.toLocalDate());

2025-06-20

syso(now.toLocalTime());

20:18:45.12346000